

**LAPORAN PRAKTIKUM
APLIKASI BERBASIS PLATFORM**

**MODUL 12 – 13 (PERTEMUAN 7)
LARAVEL DATABASE 1 & 2**



Disusun oleh:

NAJWA HUMAIRAH

2311102134

PSIF-11-REG04

Dosen Pengampu:

CAHYO PRIHANTORO, S.Kom., M.Eng

**PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO**

2026

LINK GIT :

<https://github.com/najwahmrh/Webpro/tree/main/Pertemuan%207>

Modul 12 – 13 (Pertemuan 7)

Laravel Database 1 & 2

A. Laravel

Laravel adalah kerangka kerja (framework) aplikasi web berbasis bahasa pemrograman PHP yang bersifat open-source dan mengimplementasikan arsitektur Model-View-Controller (MVC). Diciptakan oleh Taylor Otwell, Laravel dirancang untuk menyederhanakan proses pengembangan web yang kompleks menjadi lebih cepat, aman, dan elegan. Framework ini menyediakan berbagai ekosistem dan fitur built-in (bawaan) tingkat lanjut, seperti perutean (routing), manajemen basis data melalui pemetaan relasional objek (Eloquent ORM), sistem template engine (Blade), serta Command Line Interface (CLI) bernama Artisan. Penggunaan Laravel secara signifikan mendongkrak produktivitas pengembang dengan meminimalisir penulisan kode berulang pada konfigurasi standar pengembangan web.

B. Arsitektur MVC (Model-View-Controller)

Model-View-Controller (MVC) adalah paradigma desain arsitektur perangkat lunak yang mengisolasi logika bisnis dari antarmuka pengguna, sehingga menciptakan struktur kode yang modular, skalabel, dan mudah dipelihara. Arsitektur ini membagi aplikasi ke dalam tiga pilar utama :

- Model : Lapisan abstraksi data yang merepresentasikan struktur logis dari basis data. Model bertanggung jawab penuh atas seluruh manipulasi dan transaksi back-end (CRUD), termasuk validasi, penyimpanan, serta pengambilan data dari sistem basis data.
- View : Lapisan presentasi yang mendefinisikan antarmuka grafis (User Interface). View bertugas menerima instruksi atau data terstruktur dari Controller, kemudian merendernya menjadi format visual yang dapat diinteraksikan oleh pengguna. Lapis ini umumnya direpresentasikan melalui kombinasi struktur HTML, CSS, JavaScript, atau template engine bawaan kerangka kerja.
- Controller : Lapisan intermediari (penghubung) yang memfasilitasi komunikasi antara Model dan View. Controller bertindak sebagai “otak” logika aplikasi; ia bertugas menangkap HTTP Request dari klien, memproses instruksi bisnis spesifik, berinteraksi dengan Model yang relevan untuk mengambil atau memodifikasi data, dan meneruskan hasil akhirnya untuk ditampilkan kembali oleh View.

C. CRUD (Create, Read, Update, Delete)

Operasi CRUD adalah konsep dasar dalam pengelolaan data pada sistem basis data yang mencakup empat aktivitas utama, yaitu Create, Read, Update, dan Delete. Konsep ini digunakan untuk menggambarkan bagaimana data dibuat, diakses, diubah, dan dihapus dalam sebuah sistem. Dalam implementasinya, CRUD sangat erat kaitannya dengan sistem manajemen basis data seperti MySQL dan PostgreSQL, serta sering diintegrasikan dengan API dalam pengembangan aplikasi modern menggunakan framework seperti Laravel. CRUD menjadi fondasi utama dalam pengembangan aplikasi berbasis data, baik itu website, mobile app, maupun sistem enterprise.

- 1) Create adalah operasi untuk menambahkan atau menyimpan data baru ke dalam database. Dalam konteks database, operasi ini biasanya menggunakan perintah SQL INSERT INTO. Sedangkan dalam API, operasi Create direpresentasikan dengan HTTP method POST, di mana client mengirim data ke server untuk diproses. Pada Laravel, data yang diterima akan dikelola oleh controller dan disimpan menggunakan Eloquent ORM dengan method seperti create() atau save(). Contohnya adalah saat pengguna melakukan registrasi akun, maka data pengguna tersebut akan dimasukkan ke dalam tabel database.
- 2) Read adalah operasi untuk mengambil atau menampilkan data yang tersimpan di dalam database. Dalam SQL, operasi ini menggunakan perintah SELECT, sementara dalam API menggunakan HTTP method GET. Laravel akan menangani request ini melalui route dan controller, lalu mengambil data menggunakan Eloquent seperti all(), find(), atau query builder lainnya, dan mengembalikannya dalam bentuk response (biasanya JSON). Operasi ini digunakan untuk menampilkan informasi, seperti daftar produk, data pengguna, atau riwayat transaksi.
- 3) Update adalah operasi untuk mengubah atau memperbarui data yang sudah ada di dalam database. Dalam SQL, digunakan perintah UPDATE, sedangkan dalam API biasanya menggunakan HTTP method PUT atau PATCH. Di Laravel, controller akan menerima data baru dari request, mencari data lama di database, lalu memperbaruinya menggunakan method update() atau save(). Contohnya adalah ketika pengguna mengedit profilnya, seperti mengganti nama atau email, maka data lama akan diperbarui dengan data baru.
- 4) Delete adalah operasi untuk menghapus data dari database. Dalam SQL, digunakan perintah DELETE, dan dalam API menggunakan HTTP method DELETE. Laravel akan memproses request ini melalui controller, lalu menghapus data menggunakan method delete() pada model Eloquent. Dalam praktiknya, Laravel juga menyediakan fitur soft delete, yaitu data tidak langsung dihapus secara permanen, tetapi hanya diberi tanda sebagai terhapus sehingga masih bisa dipulihkan. Operasi ini biasanya digunakan untuk menghapus data yang sudah tidak diperlukan, seperti akun pengguna atau data yang tidak valid.

D. MySQL

MySQL adalah sistem manajemen basis data relasional (RDBMS) yang dirancang untuk menyimpan, mengelola, dan mengambil data secara terstruktur menggunakan bahasa SQL (Structured Query Language). Saat ini MySQL dikembangkan dan dikelola oleh Oracle Corporation. MySQL dikenal karena performanya yang cepat dan efisien, terutama untuk aplikasi berbasis web seperti website dinamis, sistem informasi, dan aplikasi CRUD (Create, Read, Update, Delete). Salah satu keunggulan utama MySQL adalah kemudahan instalasi, konfigurasi, dan penggunaannya, sehingga sangat cocok bagi pemula maupun pengembang skala kecil hingga menengah. MySQL juga mendukung berbagai engine penyimpanan seperti InnoDB (yang mendukung transaksi dan foreign key) dan MyISAM (yang lebih ringan namun tanpa dukungan transaksi). Selain itu, MySQL memiliki fitur seperti replikasi database, indexing untuk mempercepat query, sistem keamanan berbasis user privilege, serta kompatibilitas yang luas dengan berbagai

bahasa pemrograman seperti PHP, Python, Java, dan Node.js. Karena sifatnya yang stabil dan banyak digunakan, MySQL menjadi salah satu pilihan utama dalam pengembangan aplikasi berbasis web.

E. PostgreSQL

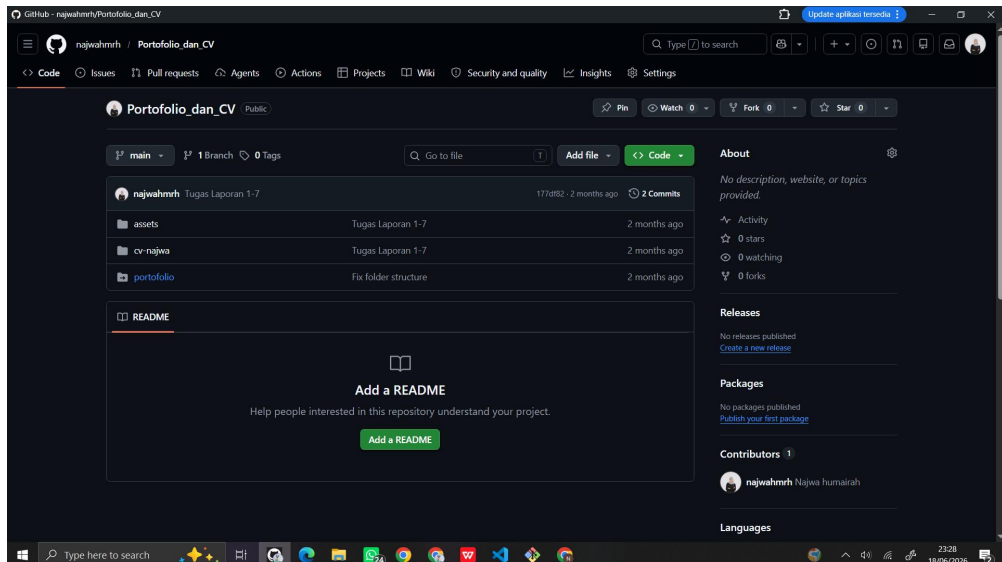
PostgreSQL adalah sistem manajemen basis data relasional open-source yang dikenal lebih kuat dan fleksibel dalam menangani data yang kompleks serta kebutuhan skala besar. PostgreSQL dikembangkan oleh komunitas global PostgreSQL Global Development Group dan sering disebut sebagai “advanced RDBMS” karena fitur-fiturnya yang sangat lengkap. PostgreSQL sangat patuh terhadap standar SQL dan mendukung konsep ACID (Atomicity, Consistency, Isolation, Durability) secara penuh untuk menjaga keandalan transaksi. Keunggulan utama PostgreSQL terletak pada kemampuannya dalam menangani tipe data yang kompleks seperti JSON, XML, array, hingga custom data type, serta dukungan terhadap query yang kompleks, stored procedure, dan trigger. PostgreSQL juga memiliki fitur indexing lanjutan (seperti GIN dan GiST), kemampuan extensibility (bisa menambahkan fungsi atau modul sendiri), serta dukungan untuk parallel query dan data partitioning yang sangat membantu dalam pengolahan data besar (big data). Karena keandalannya, PostgreSQL banyak digunakan pada aplikasi enterprise, sistem keuangan, data analytics, dan aplikasi yang membutuhkan integritas data tinggi serta skalabilitas yang baik.

Tugas :

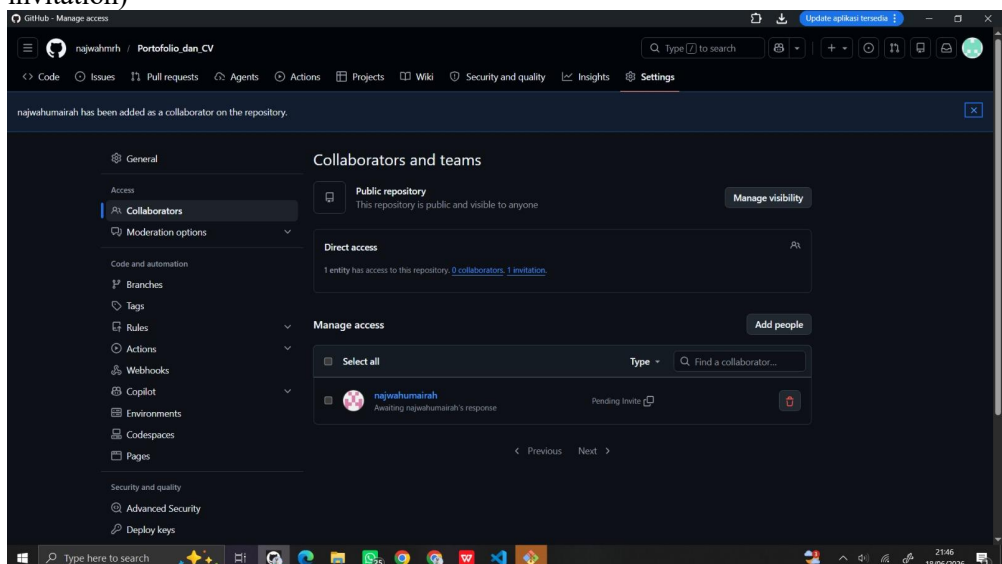
1. jelaskan tentang git branch :
 - a. Apa itu git branch
 - b. buatlah git branch dengan 2 akun berbeda dan hubungkan dengan project yang di buat di tugas 2 (bisa dengan antar teman kelas)
 - c. kalian jelaskan apa saja fungsi nya dan apa keuntungan git branch
 - d. buat juga output dan input apa saja yang dapat kalian lakukan menggunakan git branch
2. Buatlah website (bisa menggunakan website yang di gunnakan dalam tubes) , lalu tambahkan database yang terhubung dengan localhost

JAWABAN

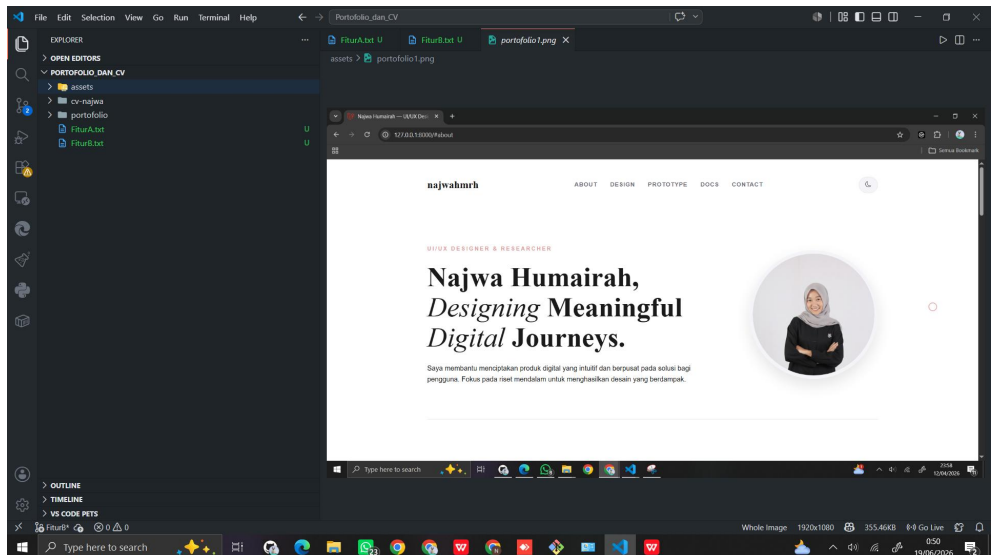
1. Berikut merupakan penjelasan tentang git branch
 - a. Git branch adalah cabang (branch) dari sebuah repository Git yang memungkinkan developer untuk mengerjakan fitur atau perubahan tanpa mengganggu kode utama (biasanya di branch main atau master).
 - b. Akun 1 : najwahmrh
 - c. Akun 2 : najwahumairah
 - Langkah 1 : Siapkan Repository
Menggunakan repository Tugas 2 Web Profile (Portofolio dan CV) yang dimiliki oleh akun 1
https://github.com/najwahmrh/Portofolio_dan_CV



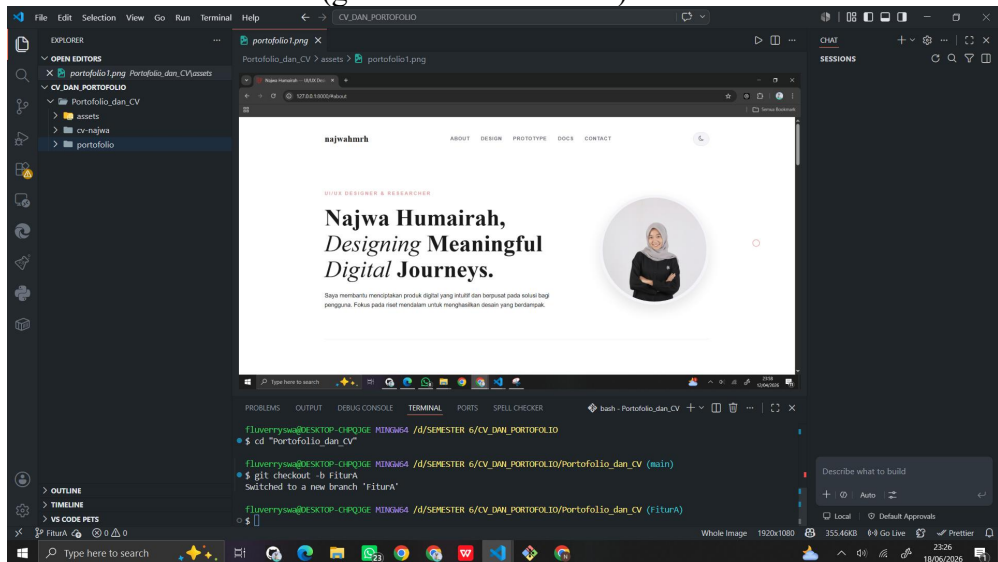
- Langkah 2 : Tambahkan akun 2 sebagai colaborator
Buka Settings – collaborators – add people – masukkan username Akun 2 – klik add to repository – klik Add [username Akun 2] (pastikan Akun 2 telah accept invitation)



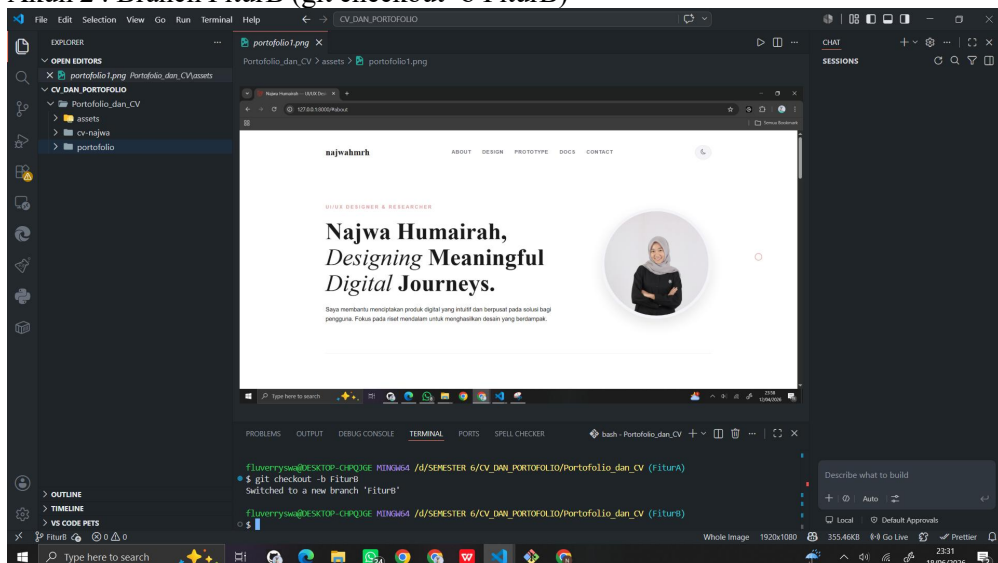
- Langkah 3 : Pada editor (VSCode) masing-masing akun, lakukan clone repository



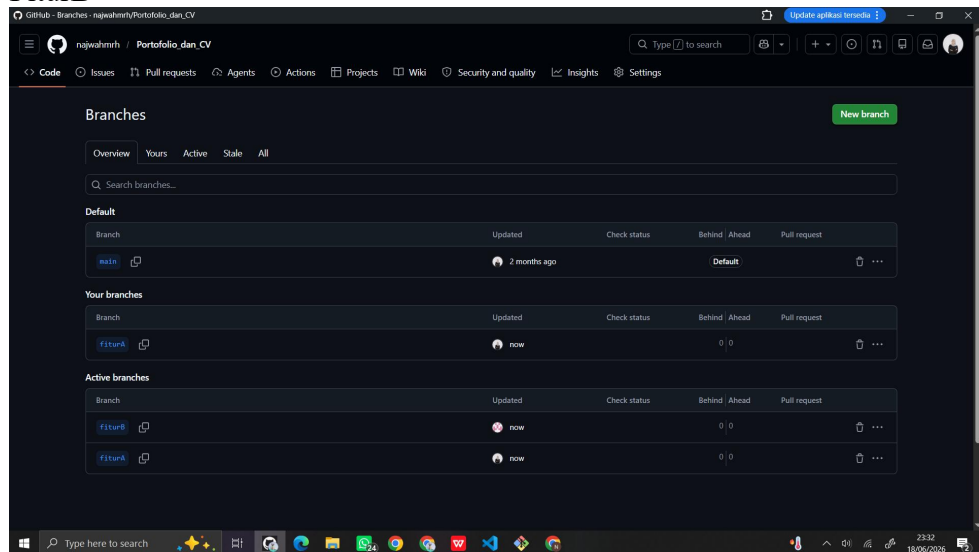
- Langkah 4 : Buat Branch untuk masing-masing akun
Akun 1 : Branch FiturA (git checkout -b FiturA)



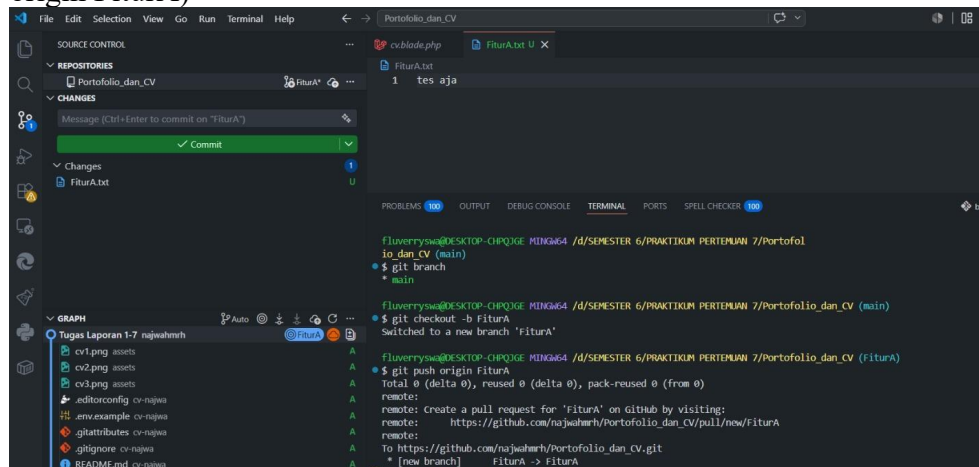
Akun 2 : Branch FiturB (git checkout -b FiturB)



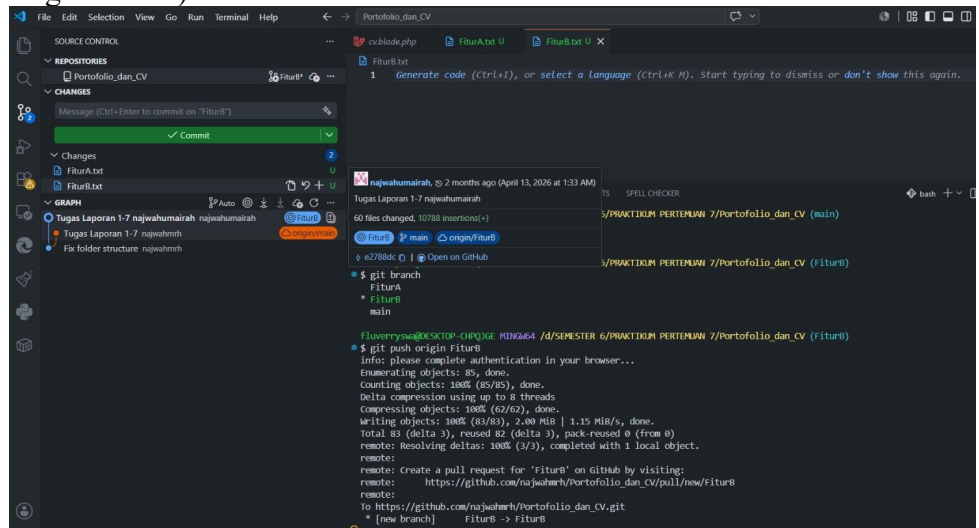
Cek pada repository bagian branches maka akan terbentuk branch FiturA dan FiturB



- Langkah 5 : Commit update fitur pada Branch masing-masing akun
Akun 1 : menambahkan file FiturA.txt dan commit ke Branch FiturA (git push origin FiturA)



Akun 2 : menambahkan file FiturB.txt dan commit ke Branch FiturB (git push origin FiturB)



- Langkah 6 : Satukan update dari kedua branch ke branch utama/branch main (dilakukan oleh Akun 1)
 Git switch main
 Git merge FiturA
 Git merge FiturB
 Git push origin main

```
Fluerry@DESKTOP-CHQJGE MINGW64 /D/SEMPER 6/PRAKTIKUM PERTEMUAN 7/Portfolio_dan_cv (main)
$ git branch
* main
  FiturA

Fluerry@DESKTOP-CHQJGE MINGW64 /D/SEMPER 6/PRAKTIKUM PERTEMUAN 7/Portfolio_dan_cv (main)
$ git checkout -b FiturB
Switched to a new branch 'FiturB'

Fluerry@DESKTOP-CHQJGE MINGW64 /D/SEMPER 6/PRAKTIKUM PERTEMUAN 7/Portfolio_dan_cv (FiturB)
$ git branch
* FiturB
  main
  FiturA

Fluerry@DESKTOP-CHQJGE MINGW64 /D/SEMPER 6/PRAKTIKUM PERTEMUAN 7/Portfolio_dan_cv (FiturB)
$ git push origin FiturB
info: please complete authentication in your browser...
Enumerating objects: 85, done.
Counting objects: 100% (85/85), done.
Delta compression using up to 8 threads
Compressing objects: 100% (62/62), done.
Writing objects: 100% (83/83), 2.00 MiB | 1.15 MiB/s, done.
Total 83 (delta 3), reused 82 (delta 3), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 1 local object.
remote:
remote: Create a pull request for 'FiturB' on GitHub by visiting:
remote:   https://github.com/najwahh/Portfolio_dan_cv/pull/new/FiturB
remote:
remote: To https://github.com/najwahh/Portfolio_dan_cv.git
   [new branch]   FiturB -> FiturB
```

- Langkah 7 : cek pada repository pada branch main maka kedua file tersebut\

d. Fungsi git branch :

- Membuat jalur pengembangan terpisah dari branch utama (main)
 - Mengembangkan fitur baru tanpa mengganggu kode utama
 - Memperbaiki bug tanpa merusak versi stabil aplikasi
 - Memungkinkan beberapa orang bekerja secara paralel dalam satu project
 - Memisahkan eksperimen atau percobaan dari kode produksi
 - Memudahkan pengelolaan versi fitur (fitur A, fitur B, dll)
- Keuntungan git branch :
- Aman untuk kode utama; main tetap stabil dan tidak mudah rusak
 - Kolaborasi lebih mudah; tiap anggota bisa punya branch sendiri

- Pekerjaan lebih terorganisir; tiap fitur punya tempat masing-masing
- Mengurangi konflik langsung di kode utama
- Memudahkan debugging; error bisa dilacak di branch tertentu
- Fleksibel untuk eksperimen; coba fitur baru tanpa risiko ke sistem utama
- Mempermudah review code sebelum digabung ke main (via merge/pull request)

e. Input & output pada git branch :

- Git branch = untuk melihat daftar branch

```
$ git branch
  FiturA
  FiturB
* main
```

- Git branch nama branch baru = untuk membuat branch baru

```
$ git branch
  FiturA
  FiturB
  branchBaru
* main
```

- Git switch nama branch = untuk pindah ke branch lain

```
$ git switch branchBaru
Switched to branch 'branchBaru'
```

- Git merge nama branch = menggabungkan branch

```
$ git merge branchBaru
Merge made by the 'ort' strategy.
  FiturA.txt | 1 +
  FiturB.txt | 1 +
  2 files changed, 2 insertions(+)
  create mode 100644 FiturA.txt
  create mode 100644 FiturB.txt
```

- Git push origin nama branch = upload branch ke remote

```
$ git push origin FiturB
info: please complete authentication in your browser...
Enumerating objects: 85, done.
Counting objects: 100% (85/85), done.
Delta compression using up to 8 threads
Compressing objects: 100% (62/62), done.
Writing objects: 100% (83/83), 2.00 MiB | 1.15 MiB/s, done.
Total 83 (delta 3), reused 82 (delta 3), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 1 local object.
remote:
remote: Create a pull request for 'FiturB' on GitHub by visiting:
remote:   https://github.com/najwahmrh/Portofolio_dan_CV/pull/new/FiturB
remote:
To https://github.com/najwahmrh/Portofolio_dan_CV.git
 * [new branch]      FiturB -> FiturB
```

- Git branch -d nama_branch = menghapus branch

```
$ git branch -d branchBaru
Deleted branch branchBaru (was 3c9108c).
```

2. Website yang saya buat adalah website yang menyimpan data biodata buku seperti judul buku, nama penulis, penerbit, jumlah halaman, dan tahun terbit. Website dibuat menggunakan framework laravel dengan database MySQL (Database bernama data_buku)

a. Source Code

- .env (koneksi ke database)

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=data_buku
DB_USERNAME=root
DB_PASSWORD=
```

Penjelasan :

Konfigurasi tersebut berfungsi sebagai pengaturan koneksi database pada file .env di framework Laravel sehingga aplikasi dapat berkomunikasi dengan database yang digunakan. Pada bagian DB_CONNECTION=mysql, ditentukan bahwa sistem menggunakan database MySQL. Selanjutnya, DB_HOST=127.0.0.1 dan DB_PORT=3306 menunjukkan bahwa database berada pada server lokal (localhost) dengan port standar MySQL, yaitu 3306.

Parameter DB_DATABASE=data_buku menunjukkan nama database yang digunakan oleh aplikasi untuk menyimpan data, khususnya data buku. Adapun DB_USERNAME=root dan DB_PASSWORD=#isi dengan password database merupakan informasi autentikasi yang diperlukan agar aplikasi dapat memperoleh akses ke database. Konfigurasi ini sangat penting karena memungkinkan Laravel menjalankan berbagai operasi pengelolaan data, seperti menambahkan, menampilkan, memperbarui, dan menghapus data (CRUD) secara optimal.

- Buku.php

```
<?php
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Buku extends Model
{
    protected $table = 'data_buku';

    protected $primaryKey = 'bukuID';
    public $incrementing = false;
    protected $keyType = 'string';
```

```

        protected $fillable = [
            'bukuID',
            'judul_buku',
            'penulis',
            'penerbit',
            'jumlah_halaman',
            'tahun_terbit'
        ];
    }

```

Penjelasan:

Kode ini merupakan Model Laravel yang merepresentasikan tabel `data_buku` dalam database guna mempermudah pengelolaan data via Eloquent ORM. Properti `protected $table = 'data_buku';` digunakan untuk mendefinisikan nama tabel secara eksplisit karena tidak mengikuti konvensi penamaan standar Laravel. Sementara itu, `protected $primaryKey = 'bukuID';` berfungsi menetapkan `bukuID` sebagai kunci utama menggantikan kolom bawaan `id`.

Karena `bukuID` bertipe string (misalnya `bk001`), fitur auto-increment dinonaktifkan melalui `public $incrementing = false;`, dan tipe datanya ditegaskan sebagai string lewat `protected $keyType = 'string';`. Terakhir, properti `$fillable` mendaftarkan atribut yang diizinkan untuk pengisian mass assignment. Pengaturan ini memastikan data seperti judul, penulis, penerbit, jumlah halaman, dan tahun terbit dapat ditambah atau diperbarui dengan aman menggunakan metode `create()` dan `update()`.

- BukuController.php

```

<?php

namespace App\Http\Controllers;

use App\Models\Buku;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;

class BukuController extends Controller
{
    public function index()
    {
        $data = Buku::all();
        if (request()->segment(1) == 'api') return
response()->json([
            'error' => false,
            'list' => $data
        ]);
        return view('buku.index', compact('data'));
    }
}

```

```

public function create()
{
    return view('buku.create');
}

public function store(Request $request)
{
    $kode = DB::transaction(function () {
        $last = Buku::orderBy('bukuID',
'desc')->lockForUpdate()->first();

        if ($last) {
            $num = (int) substr($last->bukuID, 2);
            $num++;
        } else {
            $num = 1;
        }

        return 'bk' . str_pad($num, 3, '0', STR_PAD_LEFT);
    });

    Buku::create([ 'bukuID'
        => $kode,
        'judul_buku' => $request->judul_buku,
        'penulis' => $request->penulis,
        'penerbit' => $request->penerbit,
        'jumlah_halaman' => $request->jumlah_halaman,
        'tahun_terbit' => $request->tahun_terbit,
    ]);

    return redirect()->route('buku.index');
}

public function edit($id)
{
    $buku = Buku::findOrFail($id);
    if (request()->segment(1) == 'api') return
response()->json([
        'error' => false,
        'list' => $buku
    ]);
    return view('buku.edit', compact('buku'));
}

public function update(Request $request, $id)
{
    $buku = Buku::findOrFail($id);
    $buku->update($request->all());
}

```

```

        return redirect()->route('buku.index')->with('success',
'Data berhasil diupdate');
    }

    public function destroy($id)
    {
        Buku::findOrFail($id)->delete();
        return redirect()->route('buku.index')->with('success',
'Data berhasil dihapus');
    }
}

```

Penjelasan:

Kode ini merupakan komponen Controller pada framework Laravel yang mengimplementasikan fungsi CRUD untuk entitas buku melalui model Buku. Fungsi index() mengekstraksi seluruh rekaman dari tabel data_buku untuk dirender pada view, sementara fungsi create() menyediakan antarmuka form bagi penambahan data baru.

Di dalam method store(), terdapat algoritma generasi bukuID secara otomatis dengan pola sekuensial (contoh: bk001, bk002). Proses ini mengintegrasikan DB::transaction dan lockForUpdate() demi menjaga konsistensi data dan mencegah redundansi saat terjadi konkurensi data, dilanjutkan dengan persistensi objek melalui Buku::create(). Adapun pembaruan data dikelola oleh method edit(\$id) untuk pemanggilan data lama dan update() untuk eksekusi perubahan, sedangkan eliminasi data dilakukan oleh method destroy(). Secara struktural, controller ini bertindak sebagai mediator yang memastikan seluruh instruksi dari user interface diproses dan disimpan secara valid ke database.

- index.blade.php

```

<!-- resources/views/buku/index.blade.php -->

<!DOCTYPE html>
<html>
<head>
    <title>Data Buku</title>

    <!-- Bootstrap CDN -->
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/boo
tstrap.min.css" rel="stylesheet">
</head>
<body>

```

```

<div class="container mt-5">

    <h2 class="mb-4"><img alt="book icon" data-bbox="458 121 475 138"/> Data Buku</h2>

    <a href="{{ route('buku.create') }}" class="btn btn-primary mb-3">
        + Tambah Buku
    </a>

    @if(session('success'))
        <div class="alert alert-success">
            {{ session('success') }}
        </div>
    @endif

    <div class="card shadow">
        <div class="card-body">

            <table class="table table-bordered table-striped table-hover">

                <thead class="table-dark">
                    <tr>
                        <th>Judul</th>
                        <th>Penulis</th>
                        <th>Penerbit</th>
                        <th>Halaman</th>
                        <th>Tahun</th>
                        <th width="150">Aksi</th>
                    </tr>
                </thead>

                <tbody>
                    @foreach($data as $buku)
                        <tr>
                            <td>{{ $buku->judul_buku }}</td>
                            <td>{{ $buku->penulis }}</td>
                            <td>{{ $buku->penerbit }}</td>
                            <td>{{ $buku->jumlah_halaman }}</td>
                            <td>{{ $buku->tahun_terbit }}</td>
                            <td>
                                <a href="{{ route('buku.edit', $buku->bukuID) }}" class="btn btn-warning btn-sm">
                                    Edit
                                </a>

                                <form
                                    action="{{ route('buku.destroy', $buku->bukuID) }}"
                                    method="POST" style="display:inline;">

```

```

                                @csrf
                                @method('DELETE')
                                <button type="submit"
class="btn btn-danger btn-sm"                                onclick="return
confirm('Yakin hapus data?')">                                Hapus
                                </button>
                                </form>
                                </td>
                                </tr>
                                @endforeach
                                </tbody>

                                </table>

                                </div>
                                </div>

</div>
</body>
</html>

```

Penjelasan:

Kode ini merepresentasikan halaman komponen view (index) dalam arsitektur CRUD Laravel yang ditujukan untuk memvisualisasikan seluruh data buku ke dalam format tabular. Iterasi data dari variabel \$data dieksekusi menggunakan direktif @foreach, dengan mengintegrasikan kerangka kerja Bootstrap via CDN demi menjamin responsivitas antarmuka.

Halaman ini memuat elemen navigasi berupa tombol "Tambah Buku" menuju form entri data, sekaligus mekanisme penayangan pesan keberhasilan menggunakan alert Bootstrap pasca-eksekusi manipulasi data. Selain itu, opsi "Edit" dan "Hapus" disematkan pada tiap baris data buku; fungsi destruksi data tersebut menerapkan method DELETE yang didahului oleh fitur konfirmasi untuk memvalidasi tindakan pengguna. Secara struktural, halaman ini berfungsi sebagai media interaksi utama dalam skema pemantauan dan pengelolaan data buku pada aplikasi.

- create.blade.php

```

<!-- resources/views/buku/create.blade.php -->

<!DOCTYPE html>
<html>
<head>
    <title>Tambah Buku</title>

    <!-- Bootstrap CDN -->
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/boo
tstrap.min.css" rel="stylesheet">

```



```

</head>
<body>

<div class="container mt-5">

    <h2 class="mb-4">+ Tambah Buku</h2>

    <div class="card shadow">
        <div class="card-body">

            <form action="{{ route('buku.store') }}"
method="POST">

                @csrf

                <div class="mb-3">
                    <label class="form-label">Judul
Buku</label>
                    <input type="text" name="judul_buku"
class="form-control" required>
                </div>

                <div class="mb-3">
                    <label class="form-label">Penulis</label>
                    <input type="text" name="penulis"
class="form-control" required>
                </div>

                <div class="mb-3">
                    <label class="form-label">Penerbit</label>
                    <input type="text" name="penerbit"
class="form-control" required>
                </div>

                <div class="mb-3">
                    <label class="form-label">Jumlah
Halaman</label>
                    <input type="number" name="jumlah_halaman"
class="form-control" required>
                </div>

                <div class="mb-3">
                    <label class="form-label">Tahun
Terbit</label>
                    <input type="number" name="tahun_terbit"
class="form-control" required>
                </div>

                <div class="d-flex justify-content-between">

```

```

                <a href="{{ route('buku.index') }}"
class="btn btn-secondary">
                    Kembali
                </a>

                <button type="submit" class="btn btn-
success">
                     Simpan
                </button>
            </div>

        </form>

    </div>
</div>
</div>
</body>
</html>

```

Penjelasan:

Kode ini merupakan komponen view (create) pada framework Laravel yang dialokasikan untuk proses retensi data buku baru ke dalam database. Implementasi Bootstrap melalui CDN diterapkan untuk memastikan struktur form bersifat responsif dan ergonomis bagi pengguna. Atribut data yang meliputi judul, penulis, penerbit, jumlah halaman, dan tahun terbit ditransmisikan ke route buku.store memanfaatkan method POST.

Keamanan data pada proses pengiriman dilindungi oleh direktif @csrf guna mengantisipasi celah kerentanan CSRF. Penyelarasan visual pada setiap elemen input dikendalikan secara konsisten melalui fungsionalitas class Bootstrap. Halaman ini juga mengintegrasikan tombol "Kembali" sebagai opsi navigasi ke halaman index, serta tombol "Simpan" sebagai pemicu eksekusi data. Secara struktural, halaman ini berfungsi sebagai instrumen masukan data yang valid, aman, dan sesuai dengan standar arsitektur web modern.

- edit.blade.php

```

<!-- resources/views/buku/edit.blade.php -->

<!DOCTYPE html>
<html>
<head>
    <title>Edit Buku</title>

    <!-- Bootstrap CDN -->
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/boo
tstrap.min.css" rel="stylesheet">
</head>
<body>

```

```

<div class="container mt-5">

    <h2 class="mb-4">G Edit Buku</h2>

    <div class="card shadow">
        <div class="card-body">

            <!-- ERROR VALIDATION -->
            @if ($errors->any())
                <div class="alert alert-danger">
                    <ul class="mb-0">
                        @foreach ($errors->all() as $error)
                            <li>{{ $error }}</li>
                        @endforeach
                    </ul>
                </div>
            @endif

            <form action="{{ route('buku.update',
$buku->bukuID) }}" method="POST">
                @csrf
                @method('PUT')

                <div class="mb-3">
                    <label class="form-label">Judul
Buku</label>
                    <input type="text" name="judul_buku"
class="form-control"
                        value="{{ $buku->judul_buku }}"
required>
                </div>

                <div class="mb-3">
                    <label class="form-label">Penulis</label>
                    <input type="text" name="penulis"
class="form-control"
                        value="{{ $buku->penulis }}" required>
                </div>

                <div class="mb-3">
                    <label class="form-label">Penerbit</label>
                    <input type="text" name="penerbit"
class="form-control"
                        value="{{ $buku->penerbit }}" required>
                </div>

                <div class="mb-3">

```

```

<label class="form-label">Jumlah
Halaman</label>
<input type="number" name="jumlah_halaman"
class="form-control"
value="{{ $buku->jumlah_halaman }}"
required>
</div>

<div class="mb-3">
<label class="form-label">Tahun
Terbit</label>
<input type="number" name="tahun_terbit"
class="form-control"
value="{{ $buku->tahun_terbit }}"
required>
</div>

<div class="d-flex justify-content-between">
<a href="{{ route('buku.index') }}"
class="btn btn-secondary">
Kembali
</a>

<button type="submit" class="btn btn-
primary">
<img alt="update icon" data-bbox="485 505 505 525"/> Update
</button>
</div>

</form>

</div>
</div>
</div>
</body>
</html>

```

Penjelasan:

Kode ini merupakan komponen view (edit) dalam arsitektur CRUD Laravel yang dialokasikan untuk memfasilitasi modifikasi data buku pada database. Kerangka kerja Bootstrap diintegrasikan guna menghasilkan struktur antarmuka yang rapi dan responsif. Dokumen data yang akan diperbarui ditransmisikan dari controller melalui variabel \$buku, kemudian dipetakan kembali pada komponen input menggunakan properti value.

Alur persistensi data diarahkan menuju route buku.update dengan menyertakan argumen bukuID, memanfaatkan metode POST yang dimodifikasi lewat direktif @method('PUT') demi mematuhi arsitektur RESTful Laravel. Aspek keamanan transaksi data dijamin oleh direktif @csrf untuk mengeliminasi risiko kerentanan CSRF, didukung oleh mekanisme penanganan kesalahan (error validation) yang informatif. Komponen ini ditutup dengan tombol navigasi "Kembali" dan tombol eksekusi "Update"

sebagai instrumen finalisasi perubahan data secara aman dan terstruktur.

- web.php

```
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\BukuController;

Route::get('/', [BukuController::class, 'index']);

Route::resource('buku', BukuController::class);
```

Penjelasan:

Kode tersebut merupakan konfigurasi routing pada Laravel yang berfungsi menghubungkan URL dengan controller yang sesuai. Route / diarahkan ke method index() pada BukuController, sehingga saat aplikasi diakses, halaman utama akan langsung menampilkan daftar data buku. Selain itu, penggunaan Route::resource('buku', BukuController::class); memungkinkan Laravel secara otomatis membuat seluruh route yang diperlukan untuk operasi CRUD (Create, Read, Update, Delete), sehingga proses pengelolaan data menjadi lebih praktis, terorganisir, dan tidak perlu mendefinisikan setiap route secara manual.

b. Output Cara Kerja Website

- Jalankan php artisan serve pada terminal



```
→ php artisan serve
INFO Server running on [http://127.0.0.1:8000].
Press Ctrl+C to stop the server
```

- Buka <http://127.0.0.1:8000> pada browser



Data Buku

[+ Tambah Buku](#)

Judul	Penulis	Penerbit	Halaman	Tahun	Aksi
-------	---------	----------	---------	-------	------

- Lakukan penambahan data buku (operasi create) dengan menekan tombol tambah buku & isi form, kemudian tekan tombol simpan

+ Tambah Buku

Judul Buku

Penulis

Penerbit

Jumlah Halaman

Tahun Terbit

Kembali

Simpan

- Akan diarahkan ke halaman home (index) dengan ditampilkannya data buku yang baru dimasukkan

Data Buku

[+ Tambah Buku](#)

Judul	Penulis	Penerbit	Halaman	Tahun	Aksi
Najwa	Najwa	najwahmnh	15	2026	Edit Hapus

- Lakukan edit data buku (operasi update) dengan menekan tombol edit pada salah satu baris data buku & isi form, kemudian tekan tombol update
- Akan diarahkan ke halaman home (index) dengan ditampilkannya data buku yang baru diedit

 Data Buku

[← Tambah Buku](#)

Data berhasil diupdate

Judul	Penulis	Penerbit	Halaman	Tahun	Aksi
Najwa tes edit	Najwa tes edit	najwahmrft tes edit	15	2026	Edit Hapus

- Lakukan hapus data buku (operasi delete) dengan menekan tombol hapus pada salah satu baris data buku, kemudian pada pop up yang muncul, tekan oke
- Akan diarahkan ke halaman home (index) dengan data buku yang dihapus menghilang

 Data Buku

[+ Tambah Buku](#)

Data berhasil dihapus

Judul	Penulis	Penerbit	Halaman	Tahun	Aksi
-------	---------	----------	---------	-------	------